```
version: 2.1
jobs:
  build:
    docker:
      - image: circleci/node:latest     #1
    steps:
      - checkout
      - run:
          name: Setup environment variables                        #2
          command: |                                               #2
            echo "export PACT_BROKER_TOKEN=$PACT_BROKER_TOKEN"
>> $BASH_ENV                                                       #2
            echo "export PACT_BROKER_BASE_URL=
$PACT_BROKER_BASE_URL" >> $BASH_ENV                               #2
      - run:
          name: Install project dependencies     #3
          command: npm install                   #3
      - run:
          name: Run consumer contract tests      #3
          command: npm run test:consumer         #3
      - run:
          name: Publish contract to PactFlow           #4
          command: npm run publish:pact                #4
      - run:
          name: Run provider contract tests            #4
          command: npm run start:and:test:provider     #4
```

#1 Uses the docker image for the latest node version
#2 The environment variables needed for the workflow
#3 The series of steps to run the workflow
#4 The series of steps to run the workflow

For other CI/CD platforms, it's best to check their respective documentation. However, the npm commands should remain similar, unless you change the name of the commands itself.

Now that you've had a quick overview of what a basic contract testing workflow looks like when integrated to a CI/CD pipeline, let's look at how we can improve the workflow by introducing concepts such as automated versioning, branches, can-i-deploy tool, Pact deployments, and webhooks.

10.2 Advanced features to improve the CI/CD setup

As mentioned in chapter 9, Pact has a lot of features to help you with your contract testing experience such as the *pacticipant* (a party that participates in a contract, e.g., a consumer and provider) versions, using branches,